

2026학년도 2학기 · 1주차 (9. 10.)

“지역개발 캡스톤디자인”

바이브 코딩 맛보기 — 클로드 코드로 **나의 장소 지도** 만들기

임창민

국립공주대학교 지리학과

2026. 9. 10.

■ 오늘의 순서

목차

- ① 바이브 코딩, 그 후 1년 반
- ② 준비하기 — 클로드 코드 설치
- ③ 실습 — 나의 장소 지도 만들기
- ④ 주제 다듬기 · 신청서 작성
- ⑤ 정리와 다음 시간

PART 01

바이브 코딩, 그 후 1년 반

무엇이 바뀌었고, 무엇을 배워야 하나

■ 2025년 2월, 바이브 코딩의 탄생

“요즘 제가 ‘바이브 코딩’이라 부르는 새로운 방식의 코딩을 하고 있어요. 그냥 분위기 (vibe)에 몸을 맡기고, 기술의 기하급수적 발전을 수용하며, 코드가 실제로 어떻게 생겼는지는 아예 잊어버리는 방식이죠.”

— 안드레이 카파시(Andrej Karpathy), 2025. 2. 2.
X(트위터)

요지

- 코드 직접 쓰기 ✕ → 말로 요청
- 코드 읽기 ✕ → 결과 화면만 확인
- 에러 → 메시지 그대로 붙여넣기
- “주말 프로젝트엔 꽤 쓸만하고 재미있다”

장난처럼 시작한 말 → 1년 반 뒤, 개발의 표준

■ 1년 반 동안 무엇이 바뀌었나

2025. 2

'바이브 코딩'
용어 등장
커서 붐

2025 상반기

클로드 코드
코덱스 등장
터미널형 에이전트

2025 하반기

계획 수립
파일·터미널
스스로 처리

2026 상반기

브라우저·컴퓨터
조작 · 예약 실행
원격 제어

2026. 6 → 9

Claude Fable
GPT-6 Astra
출시

2025년의 AI

코드를 써 주는 조수
붙여넣기 · 고치기 = 내 몫

2026년의 AI

계획 → 실행 → 확인하는 에이전트
내 몫 = 방향 제시 · 검증

■ 지금 이 순간의 모델들 (2026년 9월)

OpenAI

GPT-6 Astra

- 2026. 9. 4. 정식 출시
- 코딩·수학·컴퓨터 조작 최첨단
- GPT-5.6 후속 · 보안 이슈로 지연

Anthropic

Claude Fable / Mythos

- 2026. 6. 9. 출시 · Opus 상위
- 클로드 코드의 두뇌 = 우리 도구
- Fable · Opus · Sonnet · Haiku 등급

Google

Gemini

- 안티그래비티 · 제미나이 CLI
- AI 스튜디오 = 지난 학기 첫 실습

모델 이름 = 학기 중에도 바뀜 → 배울 것은 도구가 아니라 일하는 방식

출처: CNBC (2026.9.3.) "OpenAI announces rollout of GPT-6 Astra model"; Wikipedia "GPT-6 Astra"; Anthropic (2026.6.)

■ 몇 년 전 AI 코딩 vs 지금

몇 년 전 AI 코딩

“파이썬으로 지도 그리는 코드 짜 줘”

[코드 조각 제공]

[붙여넣기 → 에러] “이 에러 어떻게 고쳐?”

[수정된 코드 일부 제공]

[또 다른 문제...] 반복

지금의 바이브 코딩

“내 장소 10곳을 지도에 표시하는 웹페이지 만들어 줘”

[파일 생성 · 지도 완성 · 브라우저로 확인]

“카테고리별 색 다르게, 왼쪽에 목록도”

[전체 맥락 이해 · 수정 · 완성]

“깃허브에 올려서 공개해 줘” → URL 완성

■ 변화의 핵심: AI 에이전트의 세 가지 능력

1

계획 수립

- “공주 카페 지도 만들어 줘”
- 파일 구조 → 지도 도구 선택
- 데이터 → 화면 순서로 설계

2

도구 활용

- 파일 만들기 · 터미널 명령
- 패키지 설치 · 웹 검색
- 깃허브 업로드까지 직접

3

반복 개선

- 실행 → 에러 → 스스로 수정
- 피드백 → 전체 맥락 반영
- 기능 추가해도 구조 유지

마법사는 아님 — 방향 제시 · 결과 확인 · 구체적 개선 요청은 사람의 몫

■ AI 코딩 도구 지형도 (2026)

유형	도구	특징
IDE형 편집기 + AI	커서 · 안티그래비티 · 키로 · 윈드서프	코드 항상 보임 지난 학기 도구(커서)
에이전트·CLI형 대화로 지시	클로드 코드 · 코덱스 · 제미나이 CLI	파일·터미널·배포 스스로 이번 학기 도구
클라우드형 웹에서 바로	러버블 · 볼트 · 레플릿 · v0 · AI 스튜디오	설치 없음 · 빠른 시제품 확장성 한계

공통점

말로 요청 → 앱 완성
결과를 가르치는 것 = 요청하는 방식

차이점

얼마나 스스로 하는가
2026년의 흐름 = 에이전트형

■ 카파시 2026: 바이브 코딩 → 에이전틱 엔지니어링

“Vibe coding raises the floor.
Agentic engineering is about extrapolating the ceiling.”

— 카파시, 2026년 (세쿼이아 AI Ascent 발언 · X 게시물)

바이브 코딩 (2025)

- AI 결과 대체로 신뢰
- 시제품 문턱 ↓ · 누구나 시작
- “바닥을 올린다”

에이전틱 엔지니어링 (2026)

- AI = 틀릴 수 있는 동료 · 검증 기본
- 명세 · 검토 · 평가 루프 · 권한 관리
- “천장을 밀어 올린다”

우리 수업: 바이브 코딩으로 시작 → 에이전틱 엔지니어링으로 완성

출처: StartupHub.ai (2026.8.16.) “Andrej Karpathy: Vibe Coding to Agentic Engineering”; MindStudio, “Vibe Coding vs Agentic Engineering”

■ 카파시의 네 가지 실천 → 우리 수업 버전

1

만들기 전에 명세부터

“코드 쓰기 전에 계획만 먼저”

= 클로드 코드 Plan 모드

2

결과를 눈으로 검증

브라우저에서 열어 보기

마커 위치 · 글자 · 링크 = 내가 확인

3

확인 루프 만들기

고칠 때마다 다시 열어 보기

“이전 기능도 그대로인지 확인해 줘”

4

권한은 필요한 만큼만

파일 삭제 · 명령 실행 전 승인 창

읽고 누르기 · “항상 허용” 신중

카파시: 넷 중 “평가(검증) 설계”가 가장 덜 발달한 기술 — 출처: 앞 장과 동일

■ 좋은 요청의 다섯 요소

역할

“너는
웹 지도 개발자”

맥락

“지리학과 과제,
내 장소 소개”

목표

“장소 12곳
지도 한 페이지”

제약

“HTML 한 파일,
Leaflet, 한글”

출력

“index.html +
여는 방법”

다섯 요소를 합치면

너는 웹 지도 개발자야. 지리학과 3학년 과제로 내가 좋아하는 공주의 장소 12곳을 소개하는 지도 페이지를 만들려고 해. HTML 파일 하나(index.html)로, Leaflet 지도 위에 아래 표의 장소를 마커로 표시하고 클릭하면 이름과 설명이 뜨게 해 줘. 다 만들면 브라우저에서 여는 방법도 알려 줘.

다섯 개 전부는 불필요 — **목표 + 제약** 두 개만 분명해도 결과가 달라짐

■ 나쁜 프롬프트 vs 좋은 프롬프트

× 이렇게 하면

“지도 만들어 줘”

어디? 무슨 장소? 어떤 파일?

→ AI가 마음대로 결정

○ 이렇게

“공주 시내 내 장소 12곳을 Leaflet 지도에 마커로 표시하는 index.html 만들어 줘. 장소 표는 아래.”

범위 · 도구 · 결과 파일 명확

× 이렇게 하면

“안 돼. 고쳐 줘”

무엇이 안 되는지 AI는 모름

○ 이렇게

“지도가 회색으로만 나오고 마커가 안 보여. 브라우저 콘솔 에러: (붙여넣기)”

증상 + 에러 메시지 그대로

나쁜 프롬프트의 공통점 = 내 머릿속에만 있는 정보를 말하지 않음

■ 오늘 바로 쓰는 여섯 가지 습관

1

한 번에 하나씩

기본 지도 → 확인 → 색 → 확인
→ 목록
한꺼번에 5개 = 5개 다 어긋남

2

예시를 보여 주기

“이 사이트처럼” 링크 · 스크린샷
· 표
말 열 마디보다 정확

3

계획 먼저 받기

“코드 쓰기 전에 계획부터 설명해
줘”
= Plan 모드

4

에러는 그대로

빨간 글씨 요약 ×
복사해서 통째로 붙여넣기

5

확인하고 다음

결과 확인 → 다음 요청
안 보고 쌓으면 오류 위치 불명

6

막히면 새 대화

세 번 고쳐도 안 되면 새 대화
현재 상태 요약해서 다시 시작

■ MD(마크다운)란 무엇인가 — 파일 창에 자꾸 보이는 .md

정의

- 메모장으로 열리는 순수 텍스트 파일
- 기호 몇 개(#, -, **)로 서식 표시 = 마크다운(Markdown)
- 확장자 .md · 프로그램 불필요 · 어디서나 열림

왜 지금 중요한가

- AI가 읽고 쓰기 가장 좋은 형식
- 클로드 답변 · CLAUDE.md · README · 계획서 = 전부 md
- 워크시트 · 아이디어를 md로 쓰면 → 그대로 붙여넣기

어디서 보나

- 앱 오른쪽 파일 창 → 서식 적용된 문서로 표시
- 메모장 · 터미널 → 기호 그대로
- 깃허브 → 저장소 첫 화면(README.md)

오늘 만나는 md

- 클로드가 만든 README.md (프로젝트 설명)
- 2주차 CLAUDE.md (프로젝트 규칙)
- Plan 모드 결과 = md 계획서

.md = 기호로 서식을 적은 텍스트 — 워드 파일이 아니라 AI와 주고받는 공용 메모지

■ 기호 여섯 개면 끝 — 원문과 보이는 모습

메모장에서 보면 (원문)

공주 원도심 빈 점포 지도
목표
창업 희망자가 **빈 점포 위치**를 한눈에
핵심 기능
- 빈 점포 마커
- 업종별 필터
1. 자료 수집 (9월)
2. 개발 (10월)
데이터: [공공데이터포털](https://data.go.kr)
파일: `data/stores.csv`

파일 창에서 보면 (렌더링)

공주 원도심 빈 점포 지도

목표

창업 희망자가 빈 점포 위치를 한눈에

핵심 기능

- 빈 점포 마커
- 업종별 필터
- 1. 자료 수집 (9월)
- 2. 개발 (10월)

데이터: [공공데이터포털](https://data.go.kr) · 파일: `data/stores.csv`

제목 · ## 소제목

- 목록

1. 번호 목록

굵게

`코드` · 파일명

[글자](주소) 링크

서식 버튼 대신 기호 — 어디에 붙여넣어도 깨지지 않음 · 신청서 초안도 이렇게

■ 컨텍스트 엔지니어링 — 프로젝트 설명서 CLAUDE.md

프롬프트 → 컨텍스트

- 앞 장의 md 파일 중 특별한 하나
- 프롬프트 = 이번 한 번의 요청
- 컨텍스트 = AI가 항상 알아야 할 것
- 클로드 코드 → 폴더의 CLAUDE.md를 매번 먼저 읽음
- 팀 프로젝트 → 팀원 모두의 클로드가 같은 규칙

CLAUDE.md 예시 (팀 프로젝트용)

- # 공주 원도심 빈 점포 지도
- 목적: 공주 구도심 빈 점포를 지도에 모아 창업 수요와 연결
 - 사용자: 창업 희망자, 시청 담당자
 - 기술: HTML 한 파일 + Leaflet, 데이터는 data/stores.csv
 - 규칙: 한글 UI, 모바일에서도 보이게, 파일 지우기 전에 물어볼 것
 - 배포: GitHub Pages (main 브랜치)

2주차에 팀별로 함께 작성 — 오늘은 “이런 게 있다”만 기억

■ 이번 학기는 왜 클로드 코드인가

지난 학기 — 커서(Cursor)

- 코드 편집기 안에서 AI와 대화
- 파일 · 터미널 · 설치 = 내가 처리
- Python · Node · Git 설치에 한 시간
- 배포 = 깃허브 웹에 파일 업로드

이번 학기 — 클로드 코드

- Claude 데스크톱 앱 하나 · 설치 5분
- 파일 · 명령 · 설치 = AI가 처리
- 계획(Plan) → 실행 → 검증 흐름 내장
- “깃허브에 올려 줘” 한 줄로 배포

선배 팀(베리어프리맵)의 경험 — 도구 씨름 시간 ↓ = 문제와 사용자에게 쓰는 시간 ↑

■ 웹서비스의 세 요소 — 지도 웹서비스에서는?

프런트엔드

사용자가 보는 화면

- 지도 · 마커 · 팝업 · 목록 · 버튼
- 오늘 만드는 것 = 전부
프런트엔드

백엔드

보이지 않는 처리

- 제보 저장 · 로그인 · 경로
계산
- 팀 프로젝트에서 필요할 때

데이터

정보의 저장소

- 장소 목록 · 좌표 · 사진
- 오늘 = 파일 안의 표 · 나중 =
CSV·DB

지도 타일

배경 지도 = OpenStreetMap 등에서 빌려 옴
그 위에 우리 데이터를 얹는 구조

Leaflet

웹 지도를 그려 주는 무료 라이브러리
클로드가 알아서 사용 · 이름만 기억

■ PART 1 정리 — 오늘 실습에 가져갈 세 가지

1

목표와 제약을 말하기

무엇을 · 어떤 파일로 ·

무슨 조건으로

내 머릿속 정보를 꺼내 놓기

2

한 번에 하나, 보고 다음

작게 요청 → 열어 보기 →

다음

에러는 통째로 붙여넣기

3

AI = 틀릴 수 있는 동료

계획 먼저 · 결과 검증 ·

승인 창 읽기

= 에이전틱 엔지니어링

필요한 것 = 코딩 실력이 아니라 문제를 보는 눈 + 정확히 말하는 힘

PART 02

준비하기

Claude 데스크톱 앱 설치와 첫 화면 — 하나하나 따라 하기

■ 준비물 체크리스트

계정 3개

- ☐ 구글 계정 — 있는 것 사용
- ☐ 깃허브(GitHub) 계정 — 오늘 가입
- ☐ 클로드 팀 초대 메일 — 교수 발송 · 수락만

장비 · 자료

- ☐ 노트북 — Windows · Mac 모두 가능
- ☐ 나의 장소 10곳 이상 — 이름 · 한 줄 설명
- ☐ 휴대폰 — 마지막에 내 지도 열어 보기

클로드 구독 = **학과 팀 계정** 제공 → 개인 결제 불필요 · 학기 내 사용

■ Step 1 깃허브 가입 — 내 지도를 인터넷에 올릴 곳

1. github.com → 오른쪽 위 Sign up
2. 이메일 → 비밀번호 → 사용자 이름 (username)
3. 이메일 인증 코드 입력
4. 가입 완료 → 사용자 이름 메모 (= 내 지도 주소)

사용자 이름 정할 때

- 영문 소문자 · 숫자 · 하이픈만
- 내 지도 주소 = 사용자이름.github.io/myplaces
- 취업 포트폴리오 주소 → 깔끔하게
- 예: kimgongju → kimgongju.github.io/myplaces

이미 계정 있음 → 로그인만 확인 · 비밀번호 분실 → 지금 재설정

■ Step 2 팀 초대 수락 → 클로드 로그인

1. 이메일에서 “Anthropic” 초대 메일 열기
2. Accept invitation 클릭 → claude.ai
3. 구글 계정으로 로그인 (초대받은 이메일과 동일)
4. 오른쪽 위 조직 이름 = 지리학과 팀 → 성공

안 될 때

- 메일 없음 → 스팸함 확인 → 교수에게 이메일 재전달
- 다른 계정으로 로그인됨 → 로그아웃 → 초대받은 계정
- “이미 조직에 속해 있음” → 그대로 사용

■ Step 3 Claude 데스크톱 앱 설치

1. claude.ai/download 접속
2. Windows / Mac 버전 다운로드 → 설치 (기본값)
3. 앱 실행 → Log in → 같은 구글 계정
4. 앱으로 복귀 → 채팅 화면 표시

확인 포인트

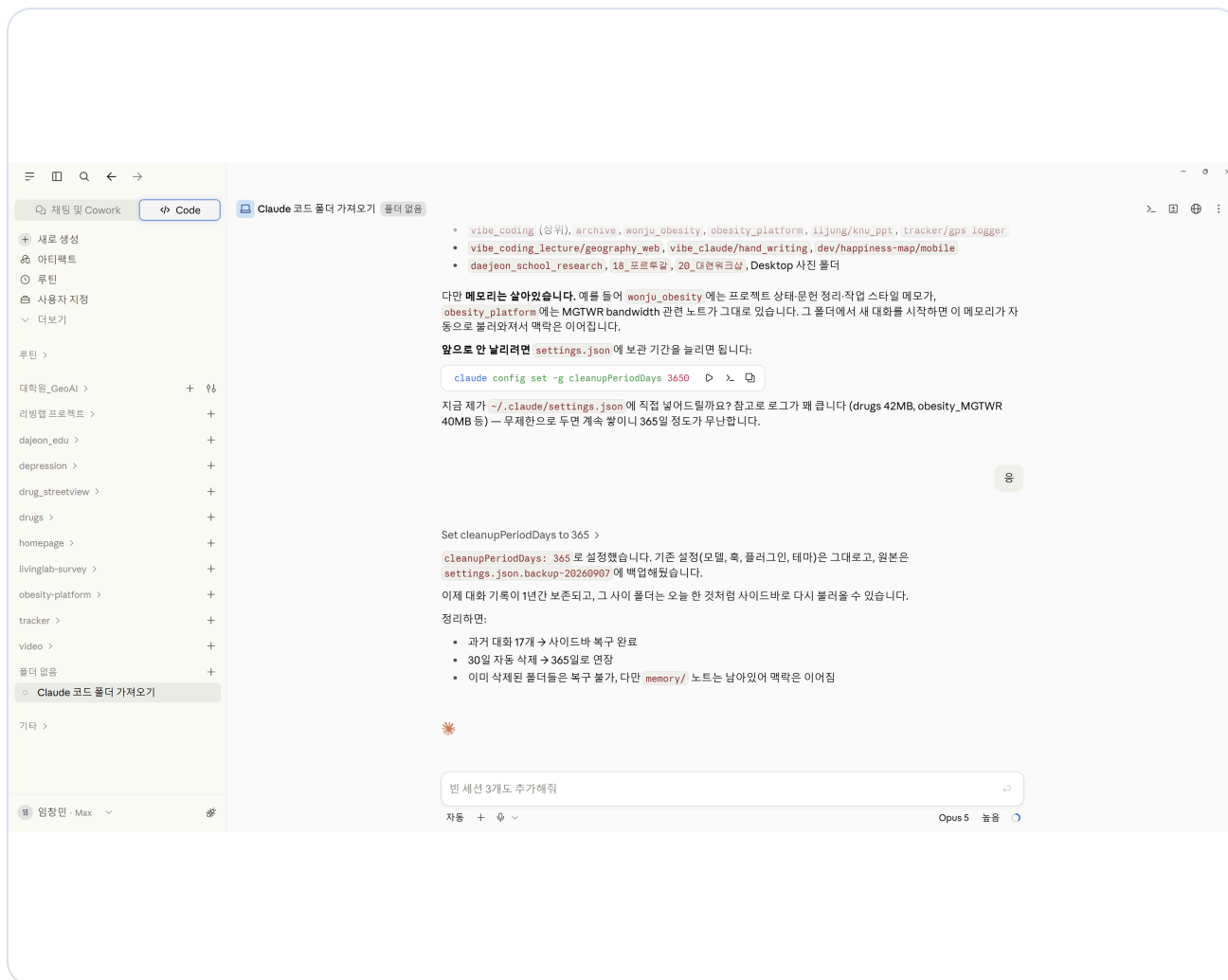
- 왼쪽 위 “채팅 및 Cowork” · “Code” 두 버튼
- 보이면 설치 끝 · 이후 진행은 앱에서

왜 앱인가

앱의 Code 탭 = 클로드 코드

터미널 · Node 설치 없이 바로 시작

■ Step 4 Code 탭으로 들어가기



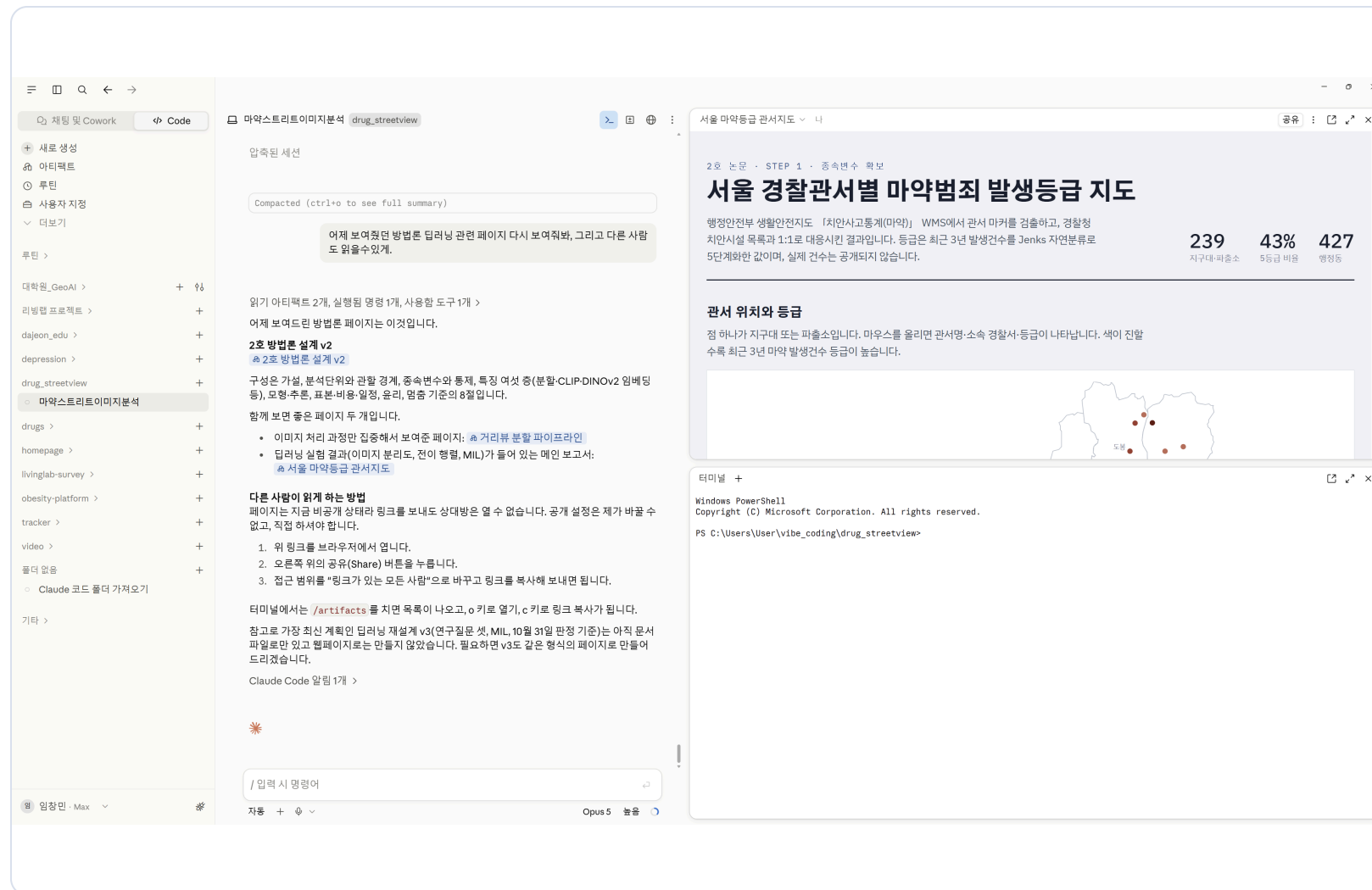
1. 왼쪽 위 </> Code 클릭

2. 사이드바 → 프로젝트 폴더
목록으로 전환

3. 맨 위 "+ 새로 생성" = 새 대화
시작 버튼

4. 아직 누르지 않기 → 다음 장에서
폴더부터

■ Step 5 화면 세 부분 이해하기



① 채팅 (가운데)

클로드에게 말하는 곳
아래 입력창에 요청 → Enter

② 파일 · 미리보기 (오른쪽 위)

만들어진 파일 · 결과 화면을 보는 곳
.md 파일 = 서식 있는 문서로 미리보기

③ 터미널 (오른쪽 아래)

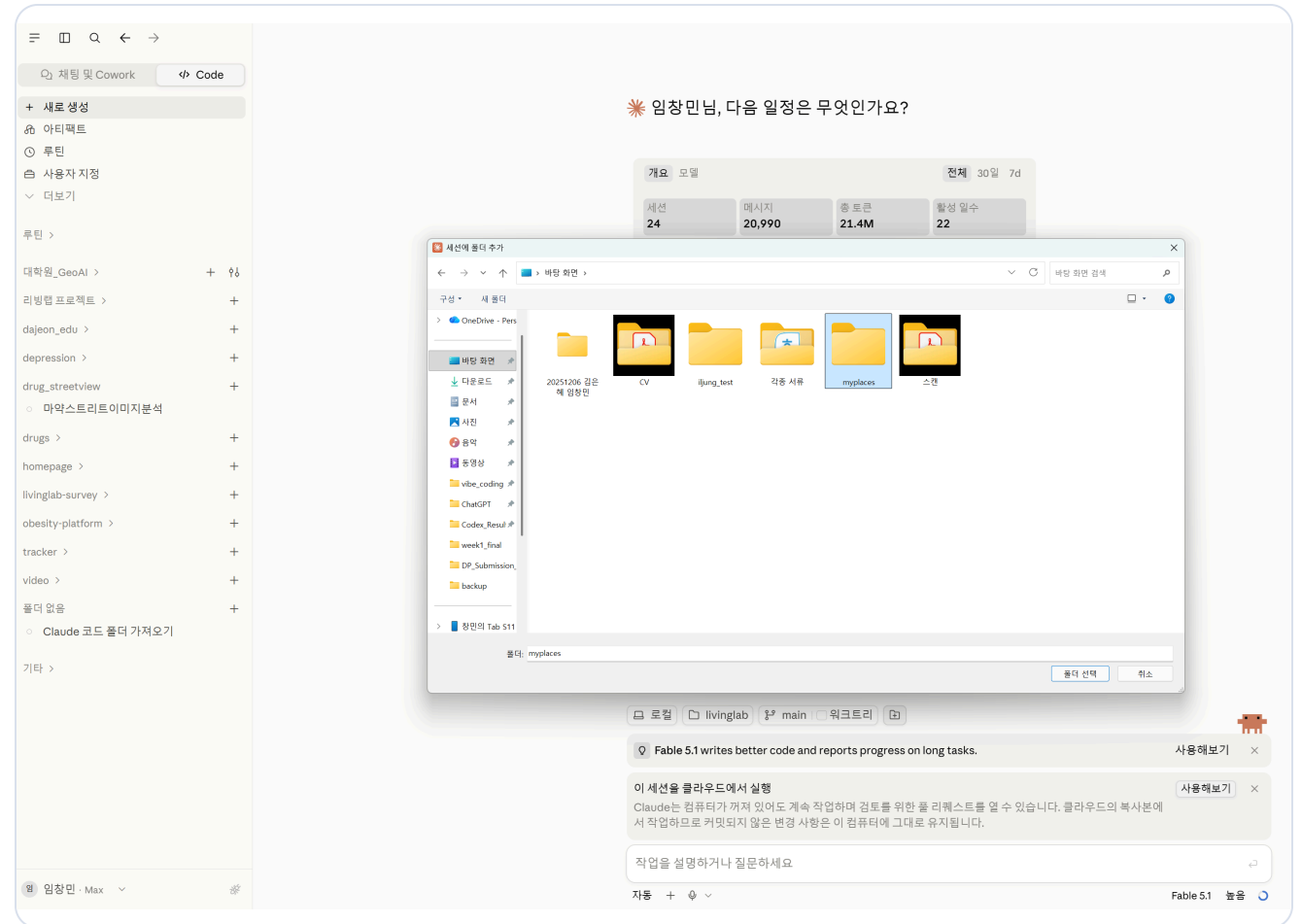
명령이 실행되는 곳
우리는 읽기만 · 직접 입력 거의 없음

■ Step 6 프로젝트 폴더 만들고 열기

1. 바탕화면 → 새 폴더 myplaces (영문
· 띄어쓰기 ✕)
2. 사이드바 맨 위 "+ 새로 생성"
3. 입력창 위 폴더 추가 아이콘 → "세션에
폴더 추가" 창
4. 바탕화면 myplaces → 폴더 선택 →
입력창 위에 myplaces 표시

폴더 = 프로젝트

**작업 범위 = 이 폴더 안만
프로젝트 1개 = 폴더 1개**



■ 알아 둘 조작 네 가지

보내기

Enter = 보내기
Shift+Enter =
줄바꿈

승인 창

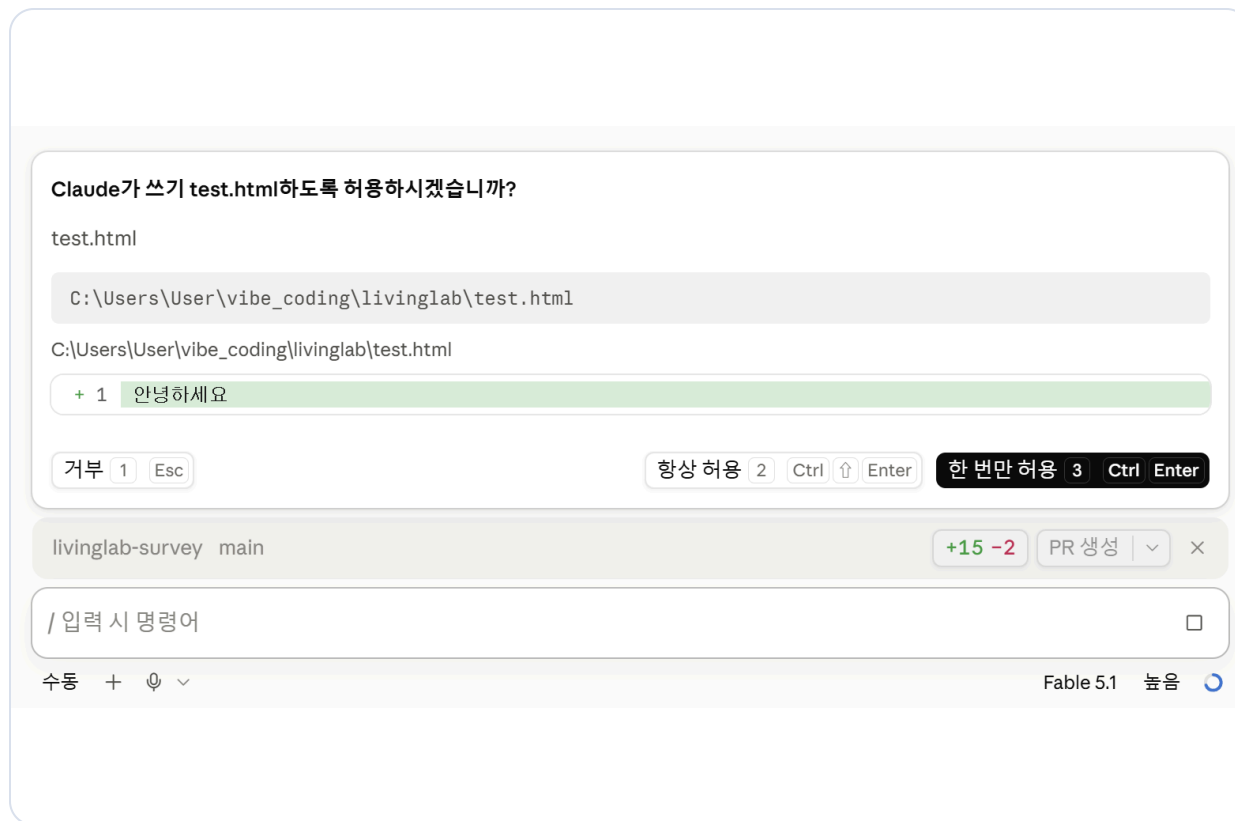
파일 쓰기 · 명령 전
질문
→ 읽고 한 번만 허용

Plan 모드

“먼저 계획만 세워 줘”
코드 전 설계 확인

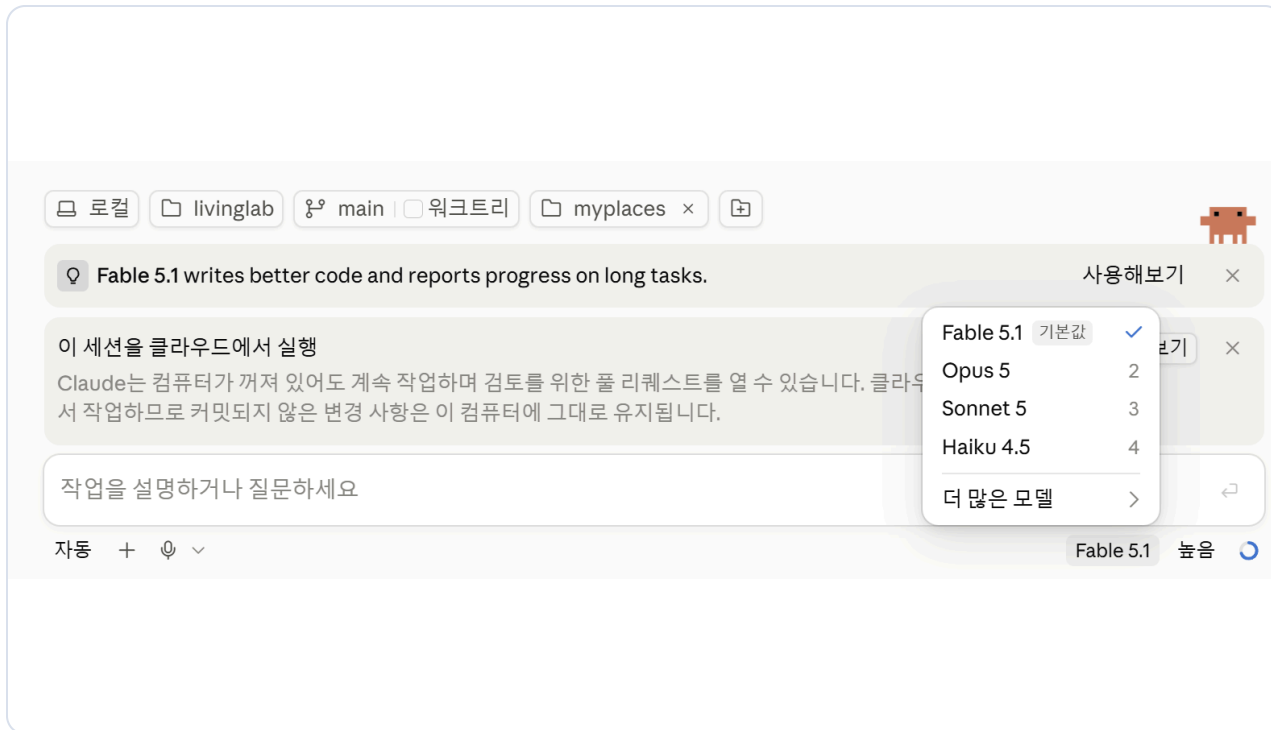
새 대화

꼬이면 새로 생성
폴더 · 파일은 그대로



버튼 셋: 거부 · 항상 허용 · **한 번만 허용** — 오늘은 **매번 읽고 한 번만 허용**하는 연습

■ 모델 고르기 — 기본값 그대로면 됩니다



기본

Fable 5.1 (기본값)

가장 똑똑한 최신 모델 → 오늘 실습 그대로 사용

막힐 때

Opus 5

세 번 고쳐도 안 될 때 → 같은 요청을 다른 모델로

빠르게

Sonnet 5 · Haiku 4.5

글자 수정 · 색 바꾸기 등 잔손질 → 빠르고 가벼움

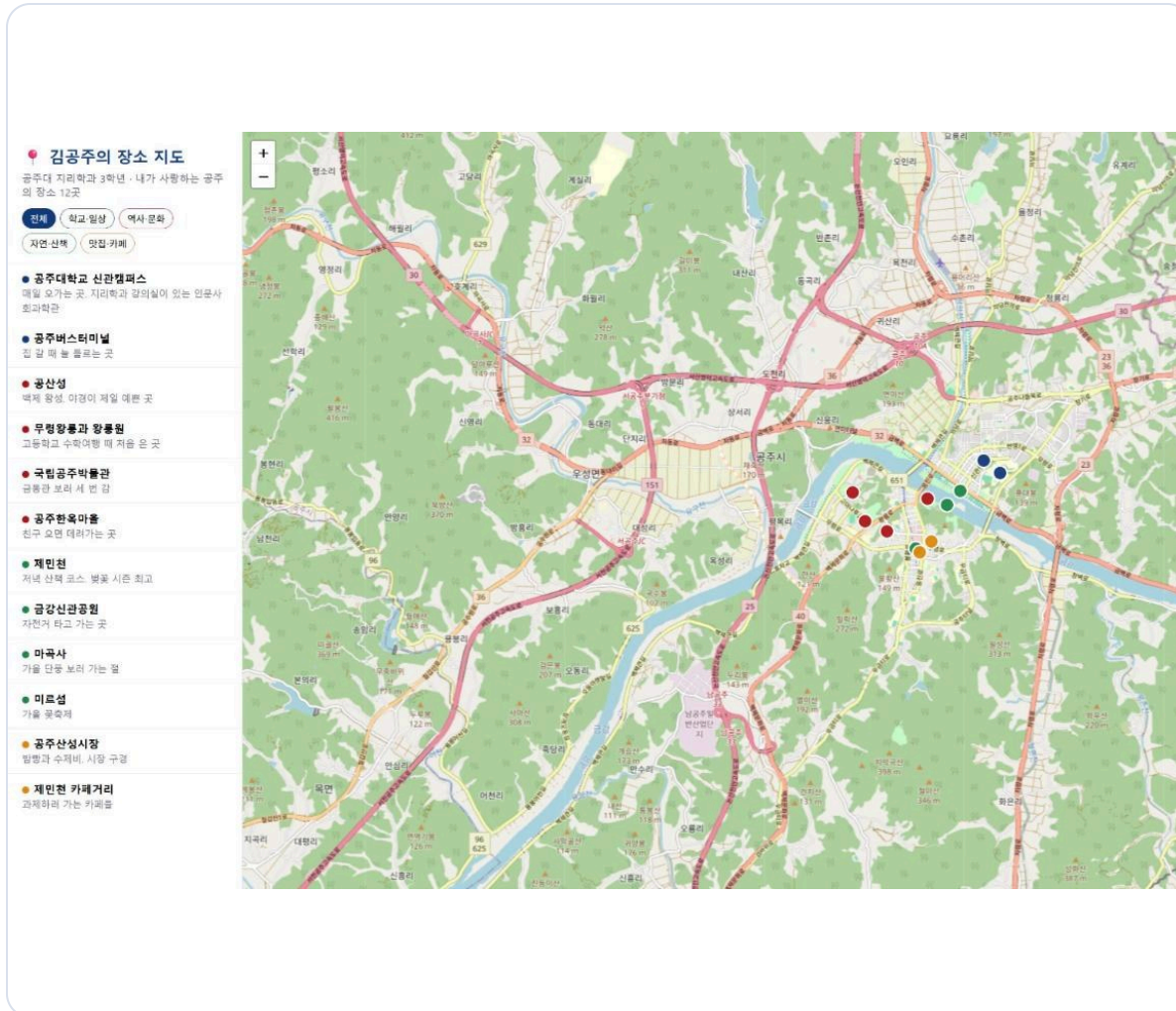
입력창 오른쪽 아래 모델 이름 클릭 → 목록 · 학기 중 이름이 바뀌어도 “기본값 = 최신” 원칙

PART 03

실습 — 나의 장소 지도

장소 10곳 이상 · 클릭하면 이야기가 뜨는 지도 · 인터넷 공개까지

■ 오늘 만들 것 — 완성 모습 미리 보기



기능

- 내 장소 10곳 이상 → 마커
- 마커 클릭 → 이름 · 카테고리 · 한 줄 이야기
- 카테고리별 색 · 필터 버튼
- 왼쪽 목록 클릭 → 지도 이동
- 인터넷 주소 → 누구나 접속

예시:

changminim.com/presentation/files/myplaces

■ 워크시트 — 나의 장소 10곳 적기 (10분)

#	장소 이름	카테고리	한 줄 이야기 (왜 이곳인가)
1	공주대 신관캠퍼스	학교·일상	매일 오가는 곳 · 지리학과 강의실
2	공산성	역사·문화	야경이 제일 예쁜 곳
3	제민천	자연·산책	저녁 산책 코스 · 벚꽃 시즌 최고
4	공주산성시장	맛집·카페	밤빵과 수제비
5			
...			
10+			

카테고리 4개 (예시)

- ● 학교·일상
- ● 역사·문화
- ● 자연·산책
- ● 맛집·카페

팁

공주가 아니어도 됨 — 고향 · 여행지 · 인생
장소

이야기가 있는 곳 = 좋은 지도

■ 좌표는 어떻게 구하나 — 두 가지 방법

방법 1 — 구글 지도에서 직접 (정확)

1. google.com/maps → 장소 검색
2. 지도 위 지점 마우스 오른쪽 클릭
3. 맨 위 숫자 36.4651, 127.1258 클릭 → 복사
4. 워크시트 옆에 붙여넣기 (위도, 경도 순)

방법 2 — 클로드에게 맡기기 (빠름 · 확인 필수)

1. 장소 이름 · 주소만 적어서 요청
2. “각 장소의 위도·경도를 찾아 표에 넣어 줘”
3. 지도 확인 → 엉뚱한 곳만 방법 1로 수정
4. 카페 · 집 등 안 유명한 곳 → 방법 1이 안전

오늘 = **방법 2로 시작** → 틀린 것만 고치기 = 에이전틱 엔지니어링의 “검증”

■ 첫 프롬프트 — 그대로 복사해서 보내기

myplaces 폴더의 새 대화에 붙여넣기 (장소 표는 내 것으로 바꾸기)

너는 웹 지도 개발자야. 지리학과 수업 과제로 “내가 좋아하는 장소 지도”를 만들려고 해.

조건: HTML 파일 하나(index.html)로 만들고, Leaflet과 OpenStreetMap 배경지도를 써. 한글로 표시해.

아래 장소들을 지도에 마커로 표시하고, 마커를 클릭하면 이름·카테고리·한 줄 이야기가 팝업으로 뜨게 해 줘.

좌표는 네가 찾아서 넣고, 확신이 없는 곳은 표시해 줘. 지도의 처음 화면은 모든 마커가 다 보이게 맞춰 줘.

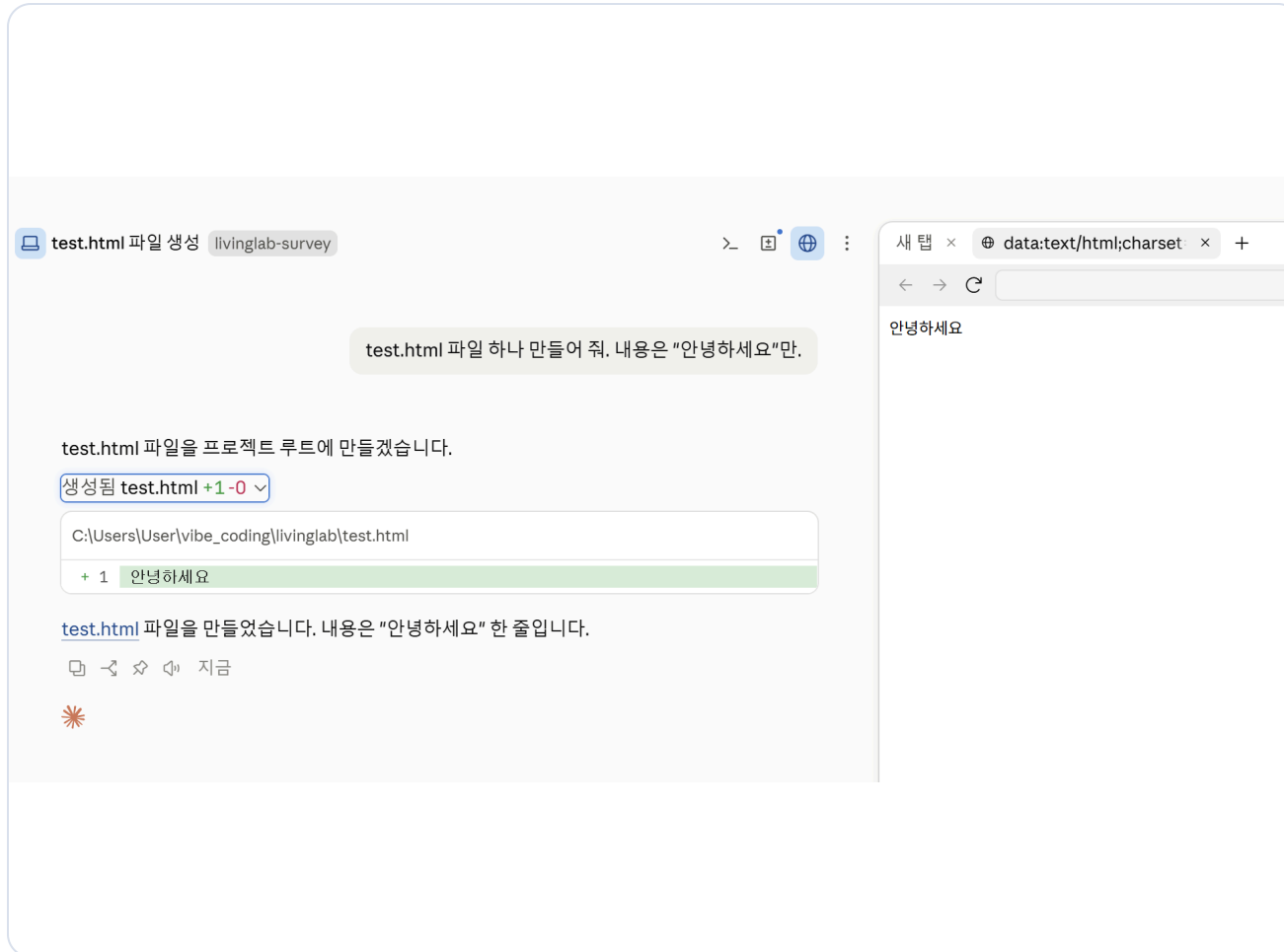
다 만들면 브라우저에서 여는 방법을 알려 줘.

장소 목록:

1. 공주대학교 신관캠퍼스 / 학교·일상 / 매일 오가는 곳, 지리학과 강의실이 있는 건물
2. 공산성 / 역사·문화 / 백제 왕성, 야경이 제일 예쁜 곳
3. 제민천 / 자연·산책 / 저녁 산책 코스, 벚꽃 시즌 최고
4. 공주산성시장 / 맛집·카페 / 밤빵과 수제비
5. ... (10곳 이상)

줄바꿈 = Shift+Enter · 보내기 = Enter — 보내기 전 장소 10곳 이상 확인

■ 보내고 나면 — 화면에 무엇이 보이나



1. "...만들겠습니다" 계획 한 줄
2. 쓰기 허용? 승인 창 → 한 번만 허용
3. 생성된 파일명 +1 표시 · 코드는 읽지 않아도 됨
4. 오른쪽 미리보기에 결과 화면 → 2단계

예시 화면

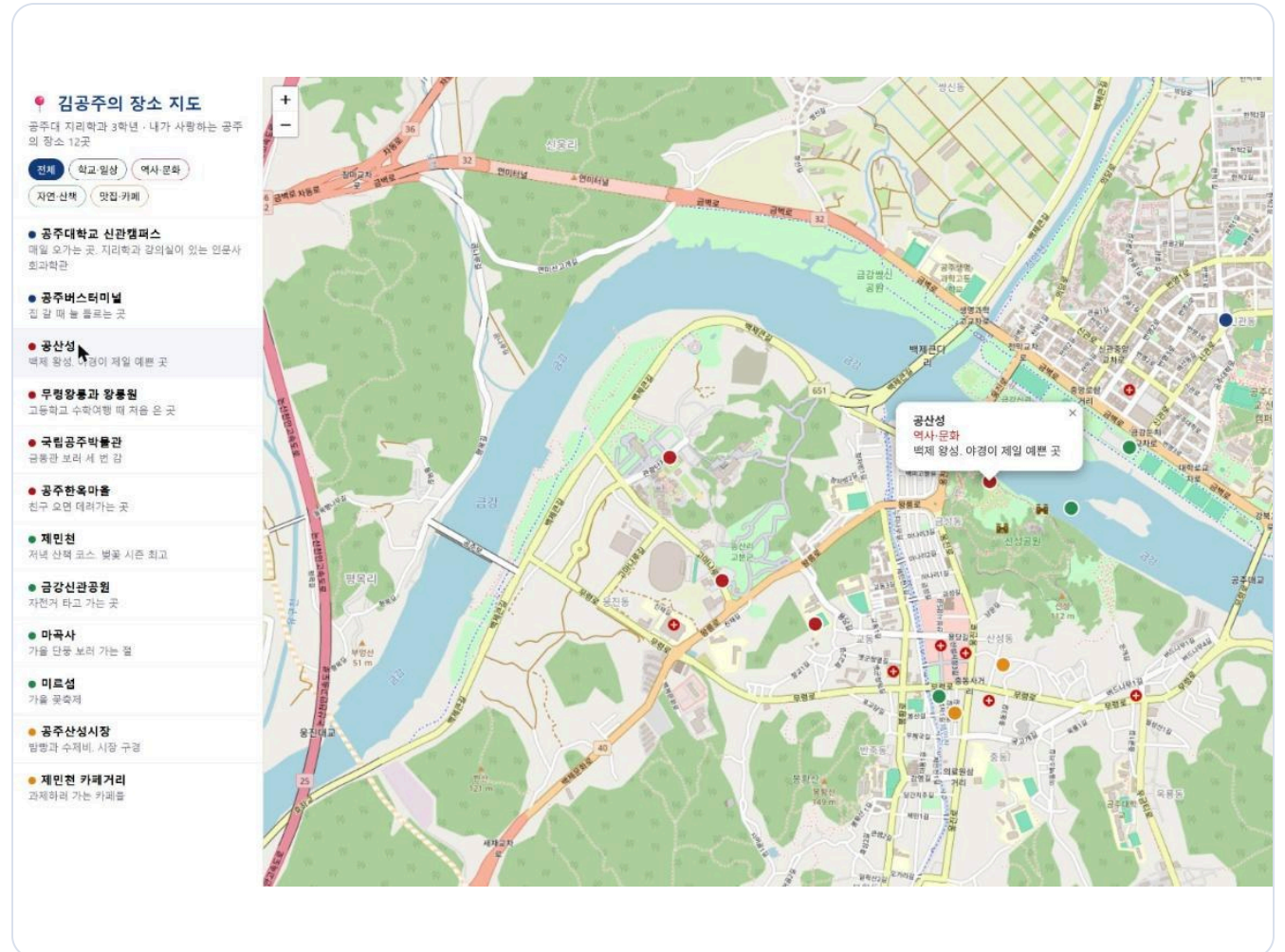
"test.html 하나 만들어 줘" 결과
실습에서는 index.html · 지도 화면이
뜸 (1~3분)

■ 결과 확인 — 브라우저에서 열기

1. 바탕화면 → myplaces 폴더
2. index.html 더블클릭 → 브라우저
3. 지도 · 마커 표시 확인
4. 마커 클릭 → 팝업에 내 이야기 확인
5. 확대 · 축소 → 엉뚱한 마커 찾기

확인 체크

- ☐ 마커 개수 = 장소 수
- ☐ 한글 깨짐 없음
- ☐ 위치 정확



■ 잘 안 되면? 자주 나오는 세 가지

지도가 회색 · 빈 화면

원인 인터넷 연결 ·
라이브러리 주소 오류
요청 "index.html을
열었는데 지도가 회색으로만
나와. F12 → Console의
빨간 글씨: (붙여넣기)"

마커가 바다 · 다른 도시에

원인 위도·경도 뒤바뀜 · AI
좌표 추정 오류
요청 "공산성 마커가 엉뚱한
곳에 있어. 정확한 좌표는
36.4651, 127.1258.
이걸로 고쳐 줘"

한글이 깨짐

원인 파일 인코딩(UTF-8)
누락
요청 "팝업의 한글이 깨져
보여. UTF-8로 고쳐 줘"

공통 규칙 = 증상 말하기 · 에러 통째로 · 정답 알면 정답도 함께

■ 개선 ① 카테고리별 색과 필터 버튼

이어서 보내기

카테고리별로 마커 색을 다르게 해 줘.

학교·일상은 남색, 역사·문화는 빨강, 자연·산책은 초록,
맛집·카페는 주황.

지도 위쪽이나 옆에 카테고리 버튼을 두고, 누르면 그
카테고리만 보이게 해 줘. “전체” 버튼도 넣어 줘.

확인할 것

- ☐ 색 = 카테고리 일치
- ☐ 버튼 클릭 → 다른 마커 사라짐
- ☐ “전체” → 복귀

원리

내가 정한 것(색 · 카테고리) = 내가 말하기
안 말하면 AI가 정함

확인 끝나기 전 다음 요청 × — **한 번에 하나**

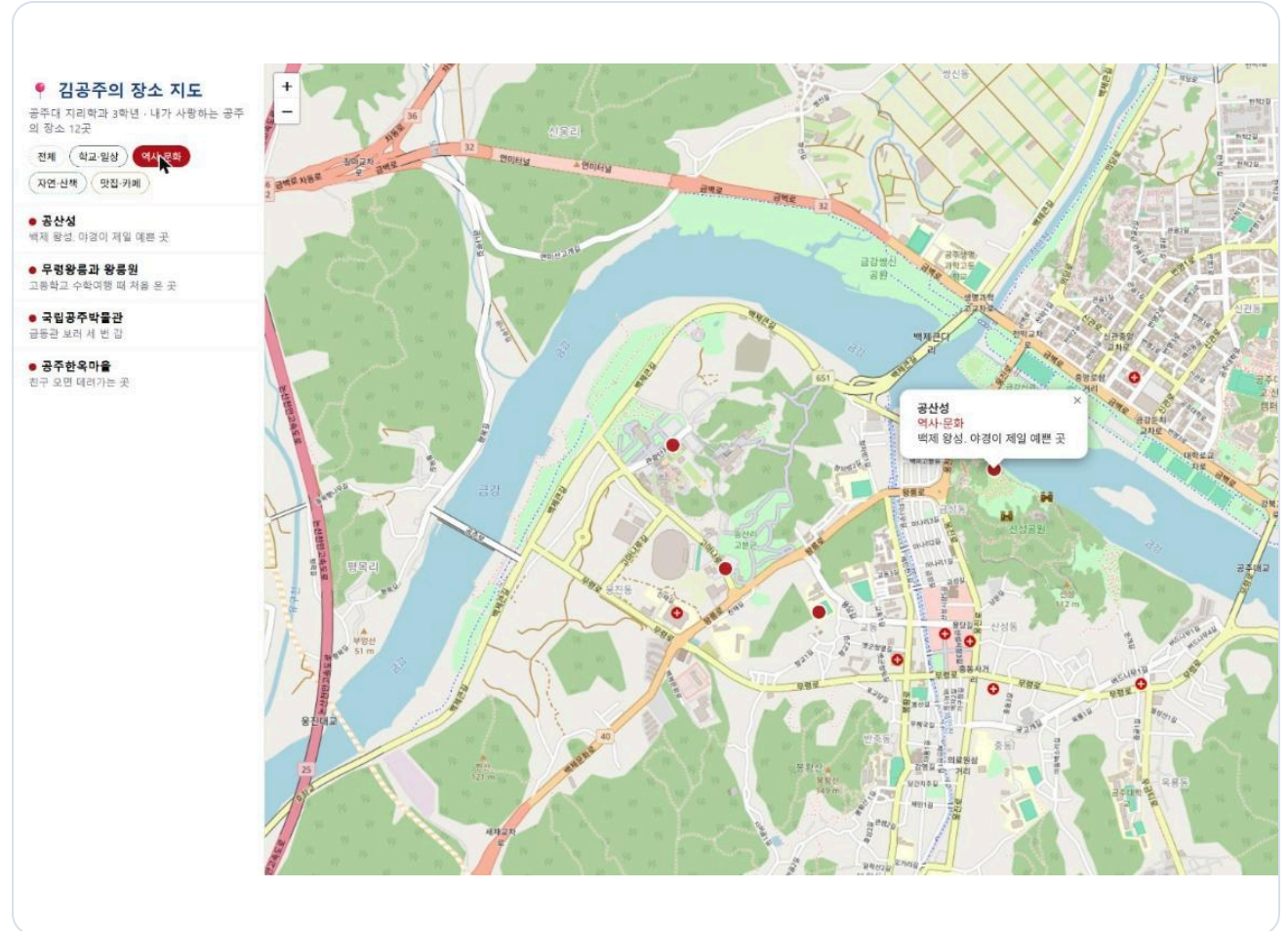
■ 개선 ② 왼쪽 목록 — 클릭하면 그곳으로

이어서 보내기

화면 왼쪽에 장소 목록 패널을 만들어 줘. 각 장소는 이름과 한 줄 이야기를 보여 주고, 클릭하면 지도가 그 장소로 부드럽게 이동하면서 팝업이 열리게 해 줘. 맨 위에는 “OOO의 장소 지도”라는 제목과 나를 소개하는 한 줄을 넣어 줘.

확인할 것

- 목록 수 = 마커 수
- 클릭 → 이동 + 팝업
- 필터 → 목록도 함께 줄어듦



■ 개선 ③ 나만의 스타일 — 사진 · 제목 · 색

골라서 보내기

- 팝업에 사진을 넣고 싶어. myplaces 폴더에 img 폴더를 만들고, 파일 이름을 장소 번호로 하면 자동으로 연결되게 해 줘.
- 전체 색을 따뜻한 베이지 톤으로 바꾸고 제목 글씨를 크게 해 줘.
- 배경 지도를 흑백(밝은 회색) 스타일로 바꿔서 마커가 더 잘 보이게 해 줘.
- 휴대폰에서 봐도 목록이 화면을 다 가리지 않게 해 줘.

사진 넣는 순서

1. myplaces 안에 img 폴더
2. 사진 이름 → 1.jpg, 2.jpg...
3. 첫 번째 요청 보내기
4. 새로고침 → 팝업 확인

주의

사진 2MB 이하 — 크면 지도 느려짐
“사진을 작게 줄여 줘” 요청 가능

■ 개선 ④ 자유 개선 — 아이디어 열두 가지

 장소 이름 검색창

 방문 날짜 · 계절 표시

 별점 · 추천 순위

 장소를 잇는 산책 코스 선

 두 장소 사이 거리 계산

 “오늘 갈 곳 랜덤 추천” 버튼

 다크 모드 전환

 내 위치에서 가장 가까운 곳

 카테고리 내 방식으로

 장소마다 사진 여러 장

 친구 댓글 (백엔드 필요!)

 마커 → 인스타 · 블로그 링크

하나 골라 요청 → 확인 → 다음 · “댓글”처럼 저장 필요 기능 = 백엔드 필요 (클로드가 설명)

■ 배포 — 인터넷에 공개하기 (기본 방법)

깃허브 웹에서 (5분 · 확실함)

1. github.com 로그인 → 오른쪽 위 + → New repository
2. 이름 myplaces · Public → Create repository
3. uploading an existing file → index.html(+img) 끌어 놓기 → Commit changes
4. Settings → Pages → Branch None → main → Save
5. 1~2분 후 새로고침 → 위쪽에 주소 표시

내 지도 주소

<https://사용자이름.github.io/myplaces/>

- 처음 404 → 2분 기다렸다 새로고침
- 파일 수정 → 다시 업로드 (같은 이름 = 덮어쓰기)
- 휴대폰으로 확인 → 친구에게 전송

■ 배포 — 클로드에게 시키기 (도전 방법)

보내기

이 폴더를 깃허브 저장소 myplaces로 만들어서 올리고, GitHub Pages로 공개해 줘. 필요한 설치나 로그인이라면 순서대로 안내해 줘. 다 되면 공개 주소를 알려 줘.

클로드가 하는 일

- Git 없음 → 설치 안내 → 승인
- 깃허브 로그인 창 안내
- 저장소 생성 → 업로드 → Pages 설정
- 주소 알려 줌

진행 중 보이는 것

1. “git이 설치돼 있는지 확인하겠습니다” → 터미널에 명령 실행
 2. 명령 실행 허용? 창 여러 번 → 읽고 한 번만 허용
 3. 브라우저 창이 열리며 깃허브 로그인 요청 → 로그인
 4. “공개 주소: <https://...github.io/myplaces>” → 열어서 확인
- 터미널 창(오른쪽 위 >_ 아이콘)을 열어 두면 진행 상황이 보임

승인 창 여러 번 → 내용 읽고 허용 · 막히면 기본 방법으로

■ URL 확인 — 내 지도가 세상에 나왔다

1

주소 열기

사용자이름.github.io/myplaces
지도 · 마커 보이면 성공

2

휴대폰으로

같은 주소를 폰에서
목록이 가리면
“모바일 대응” 요청

3

제출

주소 → 수업 제출
시트
= 오늘 실습 과제

지난 학기 웹 명함 → 오늘 지도 → 다음 달 팀의 서비스

■ 막힐 때 다섯 가지 대처

1

증상 + 에러
통째로
"...라고 떠.
화면은 이래"

2

설명 요구
"왜 안 되는지
먼저 설명해 줘"

3

되돌리기
"마지막 수정
전으로 되돌려
줘"

4

모델 바꾸기
기본값 → Opus
5
같은 요청

5

새 대화
현재 상태 요약
→ 처음부터

옆 사람에게

같은 화면을 보는 동료 = 가장 빠른 해결사
서로의 화면 보며 진행

교수에게

10분 넘게 같은 자리 → 손 들기
그 문제 = 다른 팀도 겪는 문제

PART 04

주제 다듬기 · 신청서

팀 아이디어를 신청 주제로 — 오늘 13:00 제출

■ 팀 아이디어를 클로드와 다듬기

팀 대표가 새 대화(폴더 없이도 됨)에서

우리는 지리학과 학생 3명이고, 한 학기 동안 “실제 도시 문제를 푸는 지도 웹서비스”를 만드는 캡스톤 수업을 들어.

지금 생각한 문제: (예) 공주 원도심에 빈 점포가 많은데 어디가 비었는지 아무도 모른다.

사용자: (예) 창업을 생각하는 청년, 시청 담당자

이 아이디어를 다음 형식으로 다듬어 줘.

1) 한 문장 주제명 2) 해결할 문제(누가, 무엇이 불편한가) 3) 지도에서 무엇이 보이면 해결되는가 4) 필요한 데이터와 구할 수 있는 곳 5) 핵심 기능 3개 6) 한 학기에 가능한 범위와 위험 요소

너무 거창하면 줄이고, 비슷한 기존 서비스가 있으면 알려 줘.

결과를 보고

- “못 구하는 데이터” → 말해 주기
- 기능 3개 중 1개만 가능한지 묻기
- 사용자 = 실제 한 사람으로 구체화
- 두 번 왕복 → 신청서 초안

■ 좋은 주제의 네 가지 조건

1

진짜 문제

누군가 실제로 불편한 일

“있으면 좋겠다” × → “없어서 곤란하다” ○

2

구할 수 있는 데이터

공공데이터포털 · 시청 · 직접 조사

이번 달 안에 손에 질 수 있는가

3

지도에서 보이면 달라지는 것

위치를 보여 주면 결정이 쉬워지는가

지도 = 핵심이어야

4

한 학기 크기

11월 말 공개 기준

기능 3개 이하 · 지역 좁게 · 사용자 하나

OT 예시(빈 점포 · 귀갓길 · 대중교통 사각지대) = **참고용** — 여러분의 시선으로 찾은 문제가 최고

■ 신청서 작성 요령 — 항목별

항목	이렇게 쓴다	피할 것
과제명	“공주 원도심 빈 점포 지도 서비스” = 지역 + 대상 + 지도	“AI 기반 스마트 플랫폼” 식 추상어
과제 목표	누가 · 무엇을 · 지도로 어떻게 — 세 문장	목표 다섯 개 나열
추진 내용 · 방법	자료 수집(9월) → 화면 설계 → 개발(10월) → 공개 · 피드백(11월)	기술 이름만 나열
기대 효과	사용자 한 사람의 변화 + 지역에 남는 것	“경제 활성화” 같은 큰 말
전문가 자문 계획	막힐 지점 예상 → 누구에게 · 언제	비워 두기 (2회 40만 원 지원)
재료비	답사 교통비 · 인쇄 · 설문 사례품 / 없으면 “해당 없음”	AI 구독 · 전자기기 (불가)

클로드에게 “이 신청서 항목을 우리 주제로 채워 줘” → 초안 → 팀이 고쳐서 제출

■ 오늘 13:00 제출 — 체크리스트

제출 전 확인

- ☐ 팀원 전원 이름 · 학번 · 연락처
- ☐ 과제명 = 지역 + 대상 + 지도
- ☐ 목표 세 문장 · 기능 세 개 이하
- ☐ 일정에 11월 말 공개 포함
- ☐ 전문가 자문 2회 계획
- ☐ 파일 이름: 팀명_캡스톤신청서.hwp

제출

서식 = 수업 자료실

제출 = 지정 링크 (수업 중 안내)

선정 발표

9월 셋째 주

기억할 것

신청 주제 ≠ 확정

진행하며 얼마든지 조정

PART 05

정리와 다음 시간

■ 오늘의 정리

1

배운 것

- 바이브 코딩 → 에이전틱 엔지니어링
- 좋은 요청 5요소 · 6가지 습관

2

만든 것

- 나의 장소 지도 — 10곳+
- 색 · 필터 · 목록 · 인터넷 주소

3

낸 것

- 캡스톤 신청서 (13:00)
- 내 지도 URL (제출 시트)

오늘의 지도 한 장 → 12월에는 팀의 서비스

■ 다음 시간 (9. 17.) 예고와 과제

2주차 — 팀 프로젝트의 뼈대

- Plan 모드로 팀 서비스 설계서
- 팀 폴더 + CLAUDE.md 함께 작성
- 공공데이터 CSV → 지도에 얹기
- 깃허브로 팀원과 같은 폴더 공유

과제 (다음 시간까지)

- ☐ 내 장소 지도 URL 제출 — 미완성이면 완성해서
- ☐ 개선 ④ 아이디어 하나 이상 추가
- ☐ 팀별: 주제 관련 데이터 후보 3개 (출처 링크)
- ☐ 팀별: 서비스의 사용자 한 사람 이야기 5줄

질문 = 수업 채널 — 에러 화면 캡처 + 요청한 내용 함께 올리기

■ 출처와 참고 자료

- Karpathy, A. (2025. 2. 2.). "There's a new kind of coding I call 'vibe coding'..." X(트위터) 게시물.
- StartupHub.ai (2026. 8. 16.). "Andrej Karpathy: Vibe Coding to Agentic Engineering, 2026." — 세쿼이아 AI Ascent 2026 발언 정리.
- MindStudio (2026). "Vibe Coding vs Agentic Engineering — Karpathy's Framework."
- CNBC (2026. 9. 3.). "OpenAI announces rollout of GPT-6 Astra model." / Wikipedia, "GPT-6 Astra."
- Anthropic (2026. 6.). Claude Fable · Mythos 출시 안내; Claude Code Docs, "What's new" (2026).
- Leaflet (leafletjs.com) · OpenStreetMap (openstreetmap.org) · GitHub Pages (pages.github.com)
- 지난 학기 자료: 도시계획창의적종합설계 1·2·3차시 (2026. 3.) · 본 수업 OT (2026. 9. 3.)

최신 동향 정보 = 2026년 9월 8일 기준 — 학기 중 업데이트 예정